

the server with the correct private key, the server performs a check. If the provided key is correct, access is granted to the user; otherwise it is denied.

We can use Google Authenticator with either of the above mechanisms in order to add another layer of security, which can also be called two-way authentication for SSH access. An important point that must be borne in mind when implementing two-way authentication with SSH keys is that the minimum 'openssh' version installed on your server needs to be 6.2. If the 'openssh' version is earlier than 6.2, then it must be upgraded. This is the case with CentOS 6, for which the version of 'openssh' is 5.x, which needs to be upgraded.

Integrating Google Authenticator with SSH (TOTP and user name/password)

Let's get to the fun part now, which is implementation! In this section, I will walk you through the setup, in the course of which we will implement TOTP (Google Authentication) along with the user name/password to gain access to the server. All the steps need to be performed on the server itself and nothing is to be done at the client end.

There are certain prerequisites for installing 'Google-authenticator', which I've listed below:

- (a) *autoconf*
- (b) *automake*
- (c) *libtool*
- (d) *pam-devel*

Use your system's package manager to install these packages. For example, if you are using CentOS 6, then all the above mentioned packages can be installed using *yum*, as follows:

```
yum install autoconf automake libtool pam-devel -y
```

Once done, your server is ready for the installation of Google Authenticator. Follow the steps shown below in order to install it.

To download the package, give the following command:

```
# cd /tmp
# wget -c "https://github.com/google/google-authenticator/archive/master.zip"
```

To extract the files, use the command given below:

```
# unzip master.zip
```

 **Note:** If you do not have the unzip binary installed on your server, you can easily install it using your Linux distribution package manager.

Change the directory to the extracted one, and run the following command:

```
# cd google-authenticator-master/libpam/
```

```
# ./bootstrap.sh
```

Once the above *bootstrap* script is completed successfully, run the *configure* script, followed by *make* and *make install*:

```
# ./configure
# make && make install
```

If the architecture of your server is 64-bit, you will need to copy the PAM module into the library path manually. This can be done with the following code:

```
# cp -var /usr/local/lib/security/pam_google_authenticator.so
/usr/local/lib64/security/
```

 **Note:** If you are not able to find the *pam_google_authenticator.so* file in the */usr/local/lib/security/* directory, simply search for it by using the code given below:

```
# find / -iname "pam_google_authenticator.so"
```

The above code will reveal the location of this file, which you can then copy to the above location (*/lib64/security/*).

The next step is to simply run the *google-authenticator* binary and you should see the output given below:

```
# google-authenticator
```

```
Do you want authentication tokens to be time-based (y/n) y
https://www.google.com/chart?chs=200x200&chld=M|0&cht=qr&chl=
otpath://totp/root@localhost.localdomain%3Fsecret%3D40GXT3WS
4WBDBPUG%26issuer%3Dlocalhost.localdomain
Your new secret key is: 40GXT3WS4WBDBPUG
Your verification code is 299377
Your emergency scratch codes are:
80424629
71128987
80705796
32121438
48161927
```

```
Do you want me to update your "/root/.google_authenticator"
file (y/n) y Do you want to disallow multiple uses of the
same authentication token? This restricts you to one login
about every 30s, but it increases your chances to notice or
even prevent man-in-the-middle attacks (y/n) Do you want to
disallow multiple uses of the same authentication token? This
restricts you to one login about every 30s, but it increases
your chances to notice or even prevent man-in-the-middle
attacks (y/n) y
```

By default, tokens are good for 30 seconds and in order to compensate for possible time-skew between the client and